

Stochastic Gradient, Splitting, and Saddle Points

Optimization at the data scale

Dr. Robert Hildebrand

Grado Department of Industrial & Systems Engineering
Virginia Tech

Where we are

Tuesday. First-order methods on the deterministic side: gradient descent, Nesterov, FISTA, projected gradient, Frank-Wolfe. Each iteration uses one full gradient $\nabla f(\mathbf{x})$.

What if a full gradient is too expensive? Two scenarios where you really can't afford it:

- *Big data.* $f(\mathbf{x}) = \frac{1}{N} \sum_{i=1}^N f_i(\mathbf{x})$ for $N = 10^7$. One gradient eval = N summands. Even one iteration is too slow.
- *Streaming data.* New samples arrive in real time. You never see the “full” dataset.

The fix. Use a *stochastic estimate* of the gradient — one or a few summands at a time. Iterations get faster but noisier. Engineered correctly, this trades wall-clock for a slightly worse rate, and on huge problems the trade is overwhelmingly worth it.

Today's three threads.

1. **Stochastic gradient methods** — SGD, mini-batches, momentum, Adam, variance reduction.
2. **Operator splitting** — Douglas-Rachford and ADMM. Decomposing problems with two or more non-smooth pieces.
3. **Saddle-point preview** — min-max optimization (GANs, robust learning, Lagrangians). Where

Today's roadmap

1. **The stochastic regime** — finite-sum and online optimization. Why one full gradient is no longer free.
2. **SGD** — the algorithm, the noise picture, classical $O(1/\sqrt{k})$ rate.
3. **Step-size schedules** — diminishing rates, mini-batches, the bias/variance trade-off.
4. **Momentum and Adam** — the workhorses of neural-net training. What momentum does in the stochastic regime.
5. **Variance reduction** — SVRG, SAGA. Closing the rate gap with deterministic methods.
6. **Coding break** — SGD vs. Adam vs. L-BFGS on a real ML problem.
7. **Operator splitting** — Douglas-Rachford and ADMM (Vandenberghe ECE236C, Lec. 11).
8. **Saddle points and minimax** — where descent breaks down, extragradient as the fix.
9. **Wrap-up** — Tuesday preview (research frontier).

Roadmap

The Stochastic Regime

SGD: The Basic Algorithm

Momentum and Adam

Variance Reduction

Coding Break: SGD vs Adam vs L-BFGS

Operator Splitting

Saddle Points and Minimax Optimization

Wrap-Up

Finite-sum and online optimization

Finite-sum problems (the most common).

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) = \frac{1}{N} \sum_{i=1}^N f_i(\mathbf{x}).$$

Each f_i is the loss on a single training example $(\mathbf{a}_i, y_i)$. Examples:

- Logistic regression: $f_i(\mathbf{w}) = \log(1 + \exp(-y_i \mathbf{w}^\top \mathbf{a}_i)) + \frac{\lambda}{2} \|\mathbf{w}\|^2 / N$.
- Neural-net training: $f_i(\theta)$ is the loss on sample i for parameters θ .
- Linear regression: $f_i(\mathbf{w}) = \frac{1}{2} (\mathbf{w}^\top \mathbf{a}_i - y_i)^2$.

The cost balance.

- One full gradient $\nabla f(\mathbf{x}) = \frac{1}{N} \sum_i \nabla f_i(\mathbf{x})$ costs N per-sample evaluations.
- One “stochastic” gradient $\nabla f_{i_k}(\mathbf{x})$ for a random index i_k costs 1 per-sample evaluation.
- Mini-batch: average b samples, cost b .

The online setting. Samples arrive one at a time; you can’t store them all. SGD is the only option.

The deal. Trade an exact gradient for a noisy one. Noise costs you in convergence rate, but lets you take *many* more iterations in the same wall-clock budget. For modern ML, this trade is overwhelmingly worth it.

Bottom line. For $N = 10^7$ samples, a full gradient takes seconds; a stochastic one takes microseconds. Spending those microseconds is the right move when the resulting noise can be controlled.

Roadmap

The Stochastic Regime

SGD: The Basic Algorithm

Momentum and Adam

Variance Reduction

Coding Break: SGD vs Adam vs L-BFGS

Operator Splitting

Saddle Points and Minimax Optimization

Wrap-Up

The SGD update

Stochastic gradient descent. At iteration k , sample index $i_k \in \{1, \dots, N\}$ uniformly at random and update

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \eta_k \nabla f_{i_k}(\mathbf{x}^k).$$

For mini-batches: replace ∇f_{i_k} with $\frac{1}{b} \sum_{i \in B_k} \nabla f_i(\mathbf{x}^k)$ where B_k is a random subset of size b .

Two sources of randomness.

1. The index i_k (or batch B_k) is random.
2. The iterate $\mathbf{x}^k$ is therefore random (it depends on $i_0, \dots, i_{k-1}$).

Unbiased. Since i_k is uniform, $\mathbb{E}[\nabla f_{i_k}(\mathbf{x}) \mid \mathbf{x}] = \nabla f(\mathbf{x})$. So in expectation, an SGD step is a gradient-descent step. *But* the per-step variance is nonzero:

$$\sigma^2(\mathbf{x}) := \mathbb{E} \|\nabla f_{i_k}(\mathbf{x}) - \nabla f(\mathbf{x})\|^2 = \frac{1}{N} \sum_i \|\nabla f_i(\mathbf{x}) - \nabla f(\mathbf{x})\|^2.$$

The variance is large when the per-sample gradients disagree (the data is noisy / heterogeneous) and small when they agree.

Mini-batch reduces variance. Averaging b samples cuts the variance by b (for i.i.d. samples).

Standard recipe: pick b to balance computation cost (linear in b) against statistical noise (drops as $1/\sqrt{b}$).

The noise-ball picture

With *constant* step size η , SGD converges to a **noise ball** around the optimum, not to the optimum itself.

Theorem (informal). For μ -strongly convex L -smooth f with bounded variance σ^2 :

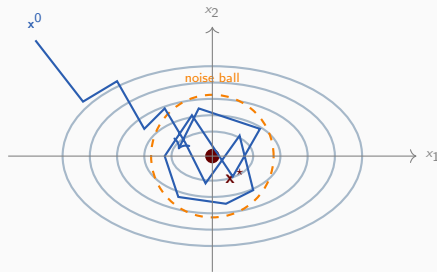
$$\mathbb{E}\|\mathbf{x}^k - \mathbf{x}^*\|^2 \leq (1 - \eta\mu)^k \|\mathbf{x}^0 - \mathbf{x}^*\|^2 + \frac{\eta\sigma^2}{\mu}.$$

Two terms: a contraction (linear rate, like GD) and a *noise-floor* term that doesn't shrink with k .

Trade-off via η .

- Large η : fast contraction, but big noise floor.
- Small η : tiny noise floor, but slow contraction.
- No constant η gives both.

The fix: *decreasing* step sizes $\eta_k \rightarrow 0$. Asymptotic rate $O(1/\sqrt{k})$ for convex, $O(1/k)$ for strongly convex.



Sketch. SGD descends quickly until it enters the noise ball of radius $\sim \sqrt{\eta\sigma^2/\mu}$. Then it bounces around indefinitely. Decreasing η_k shrinks the ball over time.

Step-size schedules and convergence rates

Robbins-Monro conditions for SGD convergence:

$$\sum_k \eta_k = \infty \quad \text{and} \quad \sum_k \eta_k^2 < \infty.$$

Step sizes must go to zero, but not too fast. The canonical choice is $\eta_k = c/k^\alpha$ for $\alpha \in (1/2, 1]$.

Rates. For convex L -smooth f with bounded variance σ^2 :

Setting	Step	Rate of $\mathbb{E}[f(\mathbf{x}^k)] - f^*$
Convex	$\eta_k = c/\sqrt{k}$	$O(1/\sqrt{k})$
μ -strongly convex	$\eta_k = c/k$	$O(1/k)$
Smooth, finite-sum, VR	— (next slide)	$O((1-c)^k)$ linear

Comparison with deterministic GD on the smooth strongly-convex case. Deterministic GD: $O((1 - 1/\kappa)^k)$ linear rate. SGD: $O(1/k)$ sublinear. *Variance reduction (next) closes this gap.*

Practical schedules.

- *Constant + decay:* keep η constant for T_0 iterations (fast initial drop), then decay as $1/k$.
- *Step decay:* divide η by 10 at preset milestones (the standard recipe in deep learning).
- *Cosine schedule:* $\eta_k = \frac{\eta_0}{2}(1 + \cos(\pi k/K))$ for K total iterations. Smooth, popular for transformer training.
- *Polyak averaging:* keep η constant but report $\bar{\mathbf{x}}^k = \frac{1}{k+1} \sum_{j=0}^k \mathbf{x}^j$. Suppresses noise without slowing the iteration.

In ML practice. The schedule matters a lot. Bad schedules can leave 1–2 orders of magnitude on the table. Modern practice mixes warmup (linear ramp at the start) with cosine or step decay.

Roadmap

The Stochastic Regime

SGD: The Basic Algorithm

Momentum and Adam

Variance Reduction

Coding Break: SGD vs Adam vs L-BFGS

Operator Splitting

Saddle Points and Minimax Optimization

Wrap-Up

SGD with momentum

Adding momentum to SGD. For a sampled stochastic gradient $\mathbf{g}^k = \nabla f_{i_k}(\mathbf{x}^k)$:

$$\begin{aligned}\mathbf{m}^{k+1} &= \beta \mathbf{m}^k + \mathbf{g}^k \\ \mathbf{x}^{k+1} &= \mathbf{x}^k - \eta_k \mathbf{m}^{k+1}\end{aligned}$$

$\mathbf{m}^k$ accumulates an exponentially-weighted average of past gradients with decay $\beta \in [0, 1)$ (typically $\beta = 0.9$).

Two interpretations.

- *Variance averaging.* Successive stochastic gradients are unbiased but noisy. Averaging cancels the noise across iterations. Effective sample size is roughly $1/(1 - \beta) \approx 10$ for $\beta = 0.9$.
- *Geometric smoothing.* Momentum approximates the smooth direction the iterates “want” to move in, ignoring per-sample fluctuations. Same intuition as Nesterov’s look-ahead.

Why momentum is standard in deep learning. Two reasons:

- Loss landscapes for neural networks have many narrow valleys / saddle points. Momentum coasts through them.
- It’s nearly free: one extra vector of memory, one extra vector add per step.

Caveat. Unlike deterministic Nesterov, stochastic momentum doesn’t have the same provable acceleration — the noise breaks the $O(1/k^2)$ argument. Practical performance is still substantially better than plain SGD on most problems.

Adam: per-coordinate adaptive momentum

Kingma & Ba (2014). Adam adapts the step size *per coordinate* using first and second moments of the gradient.

Update. Maintain $\mathbf{m}^k$ (first-moment estimate) and $\mathbf{v}^k$ (second-moment estimate). With $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$:

$$\begin{aligned}\mathbf{g}^k &= \nabla f_{i_k}(\mathbf{x}^k) \\ \mathbf{m}^{k+1} &= \beta_1 \mathbf{m}^k + (1 - \beta_1) \mathbf{g}^k \\ \mathbf{v}^{k+1} &= \beta_2 \mathbf{v}^k + (1 - \beta_2) (\mathbf{g}^k)^{\odot 2} \quad (\text{componentwise square}) \\ \hat{\mathbf{m}}^{k+1} &= \mathbf{m}^{k+1} / (1 - \beta_1^{k+1}), \quad \hat{\mathbf{v}}^{k+1} = \mathbf{v}^{k+1} / (1 - \beta_2^{k+1}) \quad (\text{bias correction}) \\ \mathbf{x}^{k+1} &= \mathbf{x}^k - \eta \hat{\mathbf{m}}^{k+1} / (\sqrt{\hat{\mathbf{v}}^{k+1}} + \varepsilon) \quad (\text{componentwise division})\end{aligned}$$

Why this works in practice.

- Coordinates with consistently large gradients (loud directions) get *smaller* effective step sizes via the $\sqrt{\hat{\mathbf{v}}}$ scaling.
- Coordinates with small / sparse gradients (quiet directions) get *larger* effective step sizes.
- This is implicit per-coordinate preconditioning, like a diagonal version of L-BFGS.

When Adam wins. Sparse gradients (NLP transformers, recommender systems), heterogeneously-scaled features, problems where the right step size differs across coordinates by orders of magnitude.

When Adam loses. Some carefully-tuned convex problems where SGD-with-momentum + a good schedule beats Adam. The adaptive scaling can hurt when the gradient noise is mostly diagonal and well-scaled. *AdamW* (decoupled weight decay) is the modern default for transformer training.

Roadmap

The Stochastic Regime

SGD: The Basic Algorithm

Momentum and Adam

Variance Reduction

Coding Break: SGD vs Adam vs L-BFGS

Operator Splitting

Saddle Points and Minimax Optimization

Wrap-Up

Closing the rate gap: variance-reduced methods

The problem. Plain SGD on μ -strongly convex problems gets $O(1/k)$. Deterministic GD on the same problems gets linear $(1 - 1/\kappa)^k$. Why the gap?

The cause. SGD's variance σ^2 doesn't go to zero as $\mathbf{x}^k \rightarrow \mathbf{x}^*$. Even at the optimum, individual gradients $\nabla f_i(\mathbf{x}^*)$ are not zero — only their average is. The constant variance forces the $O(1/k)$ rate.

Variance reduction. Use a corrected stochastic gradient whose variance *does* go to zero at the optimum:

$$\hat{\mathbf{g}}^k = \nabla f_{i_k}(\mathbf{x}^k) - \nabla f_{i_k}(\tilde{\mathbf{x}}) + \nabla f(\tilde{\mathbf{x}})$$

where $\tilde{\mathbf{x}}$ is a recently-stored “snapshot” iterate and $\nabla f(\tilde{\mathbf{x}})$ is the corresponding full gradient (computed once per epoch). The estimator is unbiased ($\mathbb{E}[\hat{\mathbf{g}}^k | \mathbf{x}^k] = \nabla f(\mathbf{x}^k)$) but with variance proportional to $\|\mathbf{x}^k - \tilde{\mathbf{x}}\|^2$, which shrinks as iterates converge.

SVRG (Johnson & Zhang 2013): uses the snapshot trick above. **SAGA** (Defazio, Bach, Lacoste-Julien 2014): caches per-sample gradient differences, avoiding the periodic full-gradient pass.

Rate. Both achieve linear convergence $O((1 - c/\kappa)^k)$ on μ -strongly convex problems — matching deterministic GD asymptotically, but at $O(1)$ stochastic gradient cost per iteration.

When to reach for VR. Finite-sum problems with $N \leq 10^7$ where you can afford the periodic full-gradient pass (SVRG) or the cache (SAGA). Less useful for online / streaming data where the full gradient is forever inaccessible.

Stochastic methods: rates summary

Iteration complexity to reach $\mathbb{E}[f(\mathbf{x}^k) - f^*] \leq \varepsilon$:

Method	Convex	μ -strongly convex
Deterministic GD	$O(1/\varepsilon)$	$O(\kappa \log(1/\varepsilon))$
Deterministic Nesterov	$O(1/\sqrt{\varepsilon})$	$O(\sqrt{\kappa} \log(1/\varepsilon))$
SGD (decreasing step)	$O(1/\varepsilon^2)$	$O(1/(\mu\varepsilon))$
SGD with momentum	similar to plain SGD	similar (no provable improvement)
SVRG / SAGA	—	$O((N + \kappa) \log(1/\varepsilon))$

Reading the table

- Plain SGD has *much* worse rate than deterministic methods. Why use it? Because each iteration is N times cheaper.
- For large N , SGD's per-iteration savings dominate; total wall-clock to a given accuracy is much smaller than deterministic GD.
- Variance reduction (SVRG/SAGA) closes the rate gap: now each stochastic iteration is $O(1)$ and the rate is linear — no compromise.

Roadmap

The Stochastic Regime

SGD: The Basic Algorithm

Momentum and Adam

Variance Reduction

Coding Break: SGD vs Adam vs L-BFGS

Operator Splitting

Saddle Points and Minimax Optimization

Wrap-Up

Coding break — what to pull up

Notebook: `part2_algorithms/23_sgd_momentum_adam.ipynb`

What we'll build:

1. **Logistic regression** on a real classification dataset ($N = 5000, p = 100$).
2. **From scratch:** SGD, SGD-with-momentum, Adam.
3. **Compare** on training loss vs. wall-clock time (not iterations) with L-BFGS as the deterministic baseline.
4. **Vary the batch size:** $b = 1, 32, 256$. See how the noise-vs-throughput trade-off plays out.
5. **Sweep step-size schedules:** constant, $1/\sqrt{k}$, step decay, cosine. Plot the difference.

What the notebook reveals. For a moderate problem ($N \leq 10^4, p \leq 100$), L-BFGS dominates. As N grows, the per-iteration cost of L-BFGS (one full gradient = $O(Np)$) eventually loses to Adam ($O(bp)$). The crossover happens around $N \approx 10^5$ on a typical laptop.

Roadmap

The Stochastic Regime

SGD: The Basic Algorithm

Momentum and Adam

Variance Reduction

Coding Break: SGD vs Adam vs L-BFGS

Operator Splitting

Saddle Points and Minimax Optimization

Wrap-Up

When you have two non-smooth pieces

The setting. $\min_{\mathbf{x}} f(\mathbf{x}) + g(\mathbf{x})$ where *both* f and g are convex but possibly non-smooth, with cheap proximal maps.

Why this is hard. ISTA/FISTA assume f is smooth. If *both* pieces are non-smooth, neither admits a gradient step.

Examples.

- $f(\mathbf{x}) = \|\mathbf{x}\|_1$, $g(\mathbf{x}) = \mathbb{I}_{\{A\mathbf{x}=b\}}$: ℓ_1 minimization with linear constraints.
- $f(\mathbf{x}) = \|A\mathbf{x} - \mathbf{b}\|_1$, $g(\mathbf{x}) = \lambda\|\mathbf{x}\|_1$: ℓ_1 regression with ℓ_1 regularization.
- $f(\mathbf{x}) = \text{TV norm}$, $g(\mathbf{x}) = \mathbb{I}_C$ (e.g. pixel-value box): TV denoising with constraints.

The trick: alternate proximal steps. If both prox_f and prox_g are cheap, build an algorithm that calls them alternately and converges to the joint minimizer. This is **operator splitting**.

Two main flavors.

- *Douglas-Rachford* (1956, originally for PDEs; rediscovered for optimization 1979): a fixed-point iteration on the reflected resolvents.
- *ADMM* (alternating direction method of multipliers): a primal-dual variant with auxiliary variables and an augmented Lagrangian.

ADMM and Douglas-Rachford are equivalent up to a change of variables (Eckstein & Bertsekas 1992).

Douglas-Rachford method

Setting. $\min f(\mathbf{x}) + g(\mathbf{x})$, both convex, both with cheap prox.

Update. Initialize $\mathbf{z}^0$. For $k = 0, 1, \dots$:

$$\begin{aligned}\mathbf{x}^{k+1} &= \text{prox}_{tg}(\mathbf{z}^k) \\ \mathbf{y}^{k+1} &= \text{prox}_{tf}(2\mathbf{x}^{k+1} - \mathbf{z}^k) \\ \mathbf{z}^{k+1} &= \mathbf{z}^k + \mathbf{y}^{k+1} - \mathbf{x}^{k+1}\end{aligned}$$

The iterate $\mathbf{z}^k$ is auxiliary (the actual approximate solution is $\mathbf{x}^k$ or $\mathbf{y}^k$).

Why this converges. Define the reflection operator $R_{tf}(\mathbf{z}) = 2\text{prox}_{tf}(\mathbf{z}) - \mathbf{z}$. Then DR is just averaged-iteration of the composition $\frac{1}{2}(I + R_{tf}R_{tg})$, which is firmly non-expansive. Standard fixed-point theory (Krasnosel'skii-Mann) gives convergence.

Rate. For convex f, g : ergodic $O(1/k)$. Linear rate under additional structure (strong convexity, Lipschitz gradient on one piece).

Practical recipe.

- Set $t \approx \sqrt{1/(L_f L_g)}$ if you have Lipschitz estimates; else tune.
- Test convergence by monitoring $\|\mathbf{x}^{k+1} - \mathbf{y}^{k+1}\|$ (this is the constraint-violation residual).
- For two-piece problems with known fast proxes, DR is often the simplest method that works.

Reading. Vandenberghe ECE236C, Lec. 11 ("Douglas-Rachford method and ADMM").

ADMM: the dual-decomposition view

Setting. $\min f(\mathbf{x}) + g(\mathbf{z})$ s.t. $A\mathbf{x} + B\mathbf{z} = \mathbf{c}$. Prototypical case: $\mathbf{z} = \mathbf{x}$, with f smooth and g non-smooth.

Augmented Lagrangian. For penalty $\rho > 0$:

$$L_\rho(\mathbf{x}, \mathbf{z}, \mathbf{u}) = f(\mathbf{x}) + g(\mathbf{z}) + \mathbf{u}^\top (A\mathbf{x} + B\mathbf{z} - \mathbf{c}) + \frac{\rho}{2} \|A\mathbf{x} + B\mathbf{z} - \mathbf{c}\|^2.$$

ADMM update.

$$\begin{aligned}\mathbf{x}^{k+1} &= \arg \min_{\mathbf{x}} L_\rho(\mathbf{x}, \mathbf{z}^k, \mathbf{u}^k) \quad (\text{subproblem in } \mathbf{x}) \\ \mathbf{z}^{k+1} &= \arg \min_{\mathbf{z}} L_\rho(\mathbf{x}^{k+1}, \mathbf{z}, \mathbf{u}^k) \quad (\text{subproblem in } \mathbf{z}) \\ \mathbf{u}^{k+1} &= \mathbf{u}^k + \rho(A\mathbf{x}^{k+1} + B\mathbf{z}^{k+1} - \mathbf{c}) \quad (\text{dual update})\end{aligned}$$

Why it's so popular. The two primal subproblems often have closed-form solutions:

- f smooth quadratic $\Rightarrow$ $\mathbf{x}$ -update is one linear solve (cached factorization).
- g a simple non-smooth (e.g., $\|\cdot\|_1$, indicator of a cone) $\Rightarrow$ $\mathbf{z}$ -update is a prox call.

You get one linear solve plus one prox per iteration — and convergence to a global optimum for convex f, g .

LASSO via ADMM. Set $f(\mathbf{x}) = \frac{1}{2} \|A\mathbf{x} - \mathbf{b}\|^2$, $g(\mathbf{z}) = \lambda \|\mathbf{z}\|_1$, with $\mathbf{x} = \mathbf{z}$. Subproblems: $\mathbf{x}$ -update is the ridge regression $(\rho I + A^\top A)^{-1}(A^\top \mathbf{b} + \rho(\mathbf{z} - \mathbf{u}/\rho))$, $\mathbf{z}$ -update is soft-thresholding.

Reading. Boyd, Parikh, Chu, Peleato, Eckstein (2011), *Distributed Optimization and Statistical Learning via ADMM*, Foundations and Trends 3(1).

Roadmap

The Stochastic Regime

SGD: The Basic Algorithm

Momentum and Adam

Variance Reduction

Coding Break: SGD vs Adam vs L-BFGS

Operator Splitting

Saddle Points and Minimax Optimization

Wrap-Up

The minimax setting — where descent breaks

Min-max problems.

$$\min_{\mathbf{x} \in X} \max_{\mathbf{y} \in Y} \Phi(\mathbf{x}, \mathbf{y}).$$

Examples.

- Lagrangian duality (we've seen these): $\Phi(\mathbf{x}, \mathbf{y}) = f(\mathbf{x}) + \mathbf{y}^\top \mathbf{g}(\mathbf{x})$.
- GANs (machine learning): $\mathbf{x}$ = generator parameters, $\mathbf{y}$ = discriminator parameters.
- Robust optimization: $\mathbf{x}$ chosen to minimize worst-case over $\mathbf{y}$ representing uncertainty.
- Two-player zero-sum games: classical min-max.

Solution concept. A point $(\mathbf{x}^*, \mathbf{y}^*)$ is a *saddle point* if

$$\Phi(\mathbf{x}^*, \mathbf{y}) \leq \Phi(\mathbf{x}^*, \mathbf{y}^*) \leq \Phi(\mathbf{x}, \mathbf{y}^*) \quad \forall \mathbf{x}, \mathbf{y}.$$

For convex-concave Φ with appropriate compactness, a saddle point exists (von Neumann minimax theorem).

The naive approach: gradient descent-ascent (GDA).

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \eta \nabla_{\mathbf{x}} \Phi(\mathbf{x}^k, \mathbf{y}^k), \quad \mathbf{y}^{k+1} = \mathbf{y}^k + \eta \nabla_{\mathbf{y}} \Phi(\mathbf{x}^k, \mathbf{y}^k).$$

On the simplest possible bilinear problem $\Phi(\mathbf{x}, \mathbf{y}) = \mathbf{x}^\top \mathbf{y}$ (saddle at origin), GDA *diverges* for any $\eta > 0$. Linearizing the update gives an eigenvalue $|1 \pm i\eta| = \sqrt{1 + \eta^2} > 1$.

The fix: extragradient. Korpelevich (1976), Nemirovski (2004). At each step, take a half-step lookahead, then use the gradient at the lookahead point:

$$\begin{aligned} \mathbf{x}^{k+1/2} &= \mathbf{x}^k - \eta \nabla_{\mathbf{x}} \Phi(\mathbf{x}^k, \mathbf{y}^k), & \mathbf{y}^{k+1/2} &= \mathbf{y}^k + \eta \nabla_{\mathbf{y}} \Phi(\mathbf{x}^k, \mathbf{y}^k), \\ \mathbf{x}^{k+1} &= \mathbf{x}^k - \eta \nabla_{\mathbf{x}} \Phi(\mathbf{x}^{k+1/2}, \mathbf{y}^{k+1/2}), & \mathbf{y}^{k+1} &= \mathbf{y}^k + \eta \nabla_{\mathbf{y}} \Phi(\mathbf{x}^{k+1/2}, \mathbf{y}^{k+1/2}). \end{aligned}$$

Two gradient evaluations per step. **Converges** on convex-concave problems at $O(1/k)$.

Modern variants. Optimistic gradient descent (Daskalakis et al. 2018), proximal point methods, and various adaptive saddle-point methods are active research.

Roadmap

The Stochastic Regime

SGD: The Basic Algorithm

Momentum and Adam

Variance Reduction

Coding Break: SGD vs Adam vs L-BFGS

Operator Splitting

Saddle Points and Minimax Optimization

Wrap-Up

Key ideas today

1. **SGD trades exact gradients for cheap iterations.** Rate slows from linear to $O(1/k)$ but per-iteration cost drops from N to 1.
2. **Momentum and Adam** are the practical backbone of large-scale ML training. Adam's per-coordinate adaptive scaling is the implicit preconditioning that makes neural-net training tractable.
3. **Variance reduction** (SVRG, SAGA) closes the rate gap on finite-sum problems.
4. **Operator splitting** (DR, ADMM) handles *two* non-smooth pieces by alternating prox calls. Universal recipe for problems with separable non-smooth structure.
5. **Saddle points** require special methods. GDA diverges; extragradient fixes it.

Tuesday preview — the research frontier.

- Distributed optimization at scale: gradient compression, federated learning.
- Modern adaptive methods: D-Adaptation, Lion, Schedule-Free SGD.
- Optimization in the LLM era: RLHF, preference learning, KL-constrained policy gradient.

Further reading

- **Bottou, Curtis, Nocedal** (2018), “Optimization methods for large-scale machine learning,” *SIAM Review* 60(2), 223–311. — the modern reference on SGD for ML.
- **Lan** (2020), *First-Order and Stochastic Optimization Methods for Machine Learning*, Springer. — definitive textbook treatment.
- **Johnson & Zhang** (2013), “Accelerating stochastic gradient descent using predictive variance reduction,” NIPS. — SVRG.
- **Defazio, Bach, Lacoste-Julien** (2014), “SAGA: A fast incremental gradient method,” NIPS.
- **Kingma & Ba** (2014), “Adam: A method for stochastic optimization,” ICLR.
- **Boyd, Parikh, Chu, Peleato, Eckstein** (2011), *Distributed Optimization and Statistical Learning via ADMM*, Foundations and Trends in ML 3(1).
- **Vandenberghe ECE236C**, Lec. 11 — Douglas-Rachford and ADMM.
seas.ucla.edu/~vandenbe/236C/lectures/dr.pdf
- **Korpelevich** (1976), “The extragradient method for finding saddle points,” *Ekonomika i Mat. Metody* 12, 747–756. — extragradient.
- **Daskalakis, Ilyas, Syrgkanis, Zeng** (2018), “Training GANs with optimism,” ICLR. — modern saddle point methods